

COMPILER DESIGN PROJECT REPORT

ErrorBuddy

A Beginner-Friendly Compiler that Reports Every Error on the Line Where the Fix Belongs, in Plain Words, with One Concrete Fix

Source code and reference repository:
https://github.com/Vardanp1/Compiler-Design-Project.git

Live website: https://vardanp1.github.io/Compiler-Design-Project/

Submitted by

Vardan Pandya
Reg. No: 24BCE2608

with teammate

Kunal Sharma
Reg. No: 24BCE2641

Under the guidance of

Prof. Boominathan P.

Index

Section	Title
1	Abstract
2	Introduction
3	Literature Survey
4	Problem Statement and Objectives
5	The ErrorBuddy Language
6	System Architecture (Architecture Diagram)
7	Phase-by-Phase Design
8	Error Reporting Subsystem
9	Diagnosis and Typo Hints
10	Sample Codes
11	Demo Screenshots
12	How to Start the Project
13	Results and Comparison with GCC
14	Limitations and Future Work
15	Conclusion
16	References

1. Abstract

Compilers are very good at deciding whether a program is valid and much less good at telling a beginner what to do when it is not. A missing semicolon on line 2 is reported by gcc on line 3 as *"expected ',' or ';' before 'printf'"*: the wrong line, parser vocabulary and no fix. Some mistakes, such as dividing by a variable that holds zero, are not reported at all. For a student, the time lost is spent not on the mistake but on decoding the message.

This project presents ErrorBuddy, a small compiler written in C++17 using only the standard library, for a C-like beginner language. It implements the classic phases (lexical analysis, recursive-descent syntax analysis, semantic analysis with a symbol table and three-address code generation) and treats error reporting as a phase of its own. Every error is a structured object with four fields: the line where the fix belongs, the phase that found it, a plain-English message and one concrete suggestion. Before reporting, the parser inspects the tokens around the failure to name the likely cause, and an edit-distance matcher suggests the intended name for typing mistakes.

The compiler is about 726 lines of C++, passes 10 of 10 automated example tests, and is published with a GitHub Pages website. In a case study of 24 classic beginner mistakes compared with gcc 11.4 (-Wall -Wextra), ErrorBuddy reported all 24 on the exact line where the fix belongs, while gcc reported 20 on the exact line and missed one mistake entirely.

Keywords: *compiler design*, *error messages*, *novice programmers*, *recursive descent*, *symbol table*, *three-address code*, *edit distance*.

2. Introduction

A compiler translates a program from a source language into a lower-level form. The classical pipeline of lexical analysis, syntax analysis, semantic analysis and intermediate code generation is designed to decide, precisely and quickly, whether a program belongs to the language. When it does not, the compiler stops and prints a diagnostic. That diagnostic is written in terms of the compiler's own grammar and reports the position where the parser noticed a problem.

For an experienced programmer this is enough. For a beginner the two weaknesses compound: the reported line is often after the line that needs editing, and the message describes what the parser expected rather than what the student should type. The fault is small and the cost is large: the student re-reads correct code on the reported line, guesses, recompiles and repeats.

An important observation motivating this project is that the information needed for a better message is already available at the moment of failure. The parser knows the previous token, the next token and the rule it was trying to complete. ErrorBuddy uses that information deliberately: diagnosis runs before the error is reported, and the report is built for the reader.

2.1 Scope of the work

- Design of a small C-like beginner language with `int` variables, arithmetic, `print` and comments.
- Implementation of a complete front end and three-address code generator in C++17 with no external tools (726 lines in five files).

- An error-reporting subsystem: the four-field `CompileError`, token-context diagnosis, fix-line selection and typo hints.
- An automated test suite (`make test`), a case study against gcc, a local web playground and a GitHub Pages website.

2.2 Team contribution

Member	Contribution
Vardan Pandya (24BCE2608) Kunal Sharma (24BCE2641)	Joint work on all parts of the project: language design, lexer, parser, semantic analysis, three-address code generation, the error-reporting subsystem, testing, the case study against gcc, the web playground and the GitHub Pages website, and this report.

3. Literature Survey

ErrorBuddy builds on established ideas in compiler construction and on research into compiler error messages for novice programmers. The following works shaped the design.

Work / tool	What it contributes	Relevance to ErrorBuddy
Aho, Lam, Sethi, Ullman: *Compilers: Principles, Techniques, and Tools*	Standard phase structure, recursive-descent parsing, symbol tables and three-address code.	Basis of the phase design and the TAC format.
Wirth: *Compiler Construction* (1996)	Hand-written recursive-descent compilers, one procedure per grammar rule.	Same structure; hand-writing keeps every message under our control.
Levenshtein (1966)	Edit distance between two strings.	Used for 'Did you mean' hints on misspelled names and keywords.
Traver (2010): On compiler error messages	Analyses why messages are hard to understand: wrong position, compiler terminology.	Targets both problems: fix-line reporting and plain English.
Marceau, Fisler, Krishnamurthi (2011)	Measures how students respond to error messages designed for novices.	Motivates measuring the messages instead of assuming they help.
Denny, Luxton-Reilly, Carpenter (2014)	Enhanced syntax messages alone did not measurably help students.	Suggests that better wording is not enough; the location and cause must be right.
Becker et al. (2016, 2019)	Enhanced messages can reduce repeated errors; survey and guidelines for error message design.	Guidelines followed: be specific, use plain language, suggest a fix.
Elm and Clang diagnostics	Production compilers with friendly, fix-it style messages.	Shows the style is practical; ErrorBuddy applies it to a teaching language.

Gap identified: much of the work on novice error messages rewrites or enhances the text after a production compiler has already chosen the line and the cause. If the line is wrong, better wording

cannot fix it. ErrorBuddy builds the diagnosis into the parser, so the line and the cause are chosen at the point of failure, and measures the result against gcc.

4. Problem Statement and Objectives

Problem statement: Compilers tell beginners that something is wrong. They do not reliably say where to fix it, or how.

Research question: How can a compiler report each beginner mistake on the line where the fix belongs, in plain words, with one concrete fix?

4.1 Objectives

- **Build a working compiler:** source text to three-address code, with every core phase implemented by hand.
- **Report on the fix line:** the line the student must edit, not the line where parsing stopped.
- **Explain in plain words:** name the broken rule without parser vocabulary.
- **Suggest one fix:** every error carries one concrete thing to type.
- **Stop broken programs:** no intermediate code is printed for a program with an error.
- **Measure it:** compare the reported lines with gcc on a hand-labelled set of mistakes.

4.2 Three mistakes and what gcc 11.4 reports

```
int a = 5;
int b = 10

// show the total
printf("%d\n", a + b);
// gcc: line 5: expected ',' or ';' before 'printf'   (fix is on line 2)
```

```
int x = 4;
int y = x + * 3;
printf("%d\n", y);
// gcc: line 2: invalid type argument of unary '*' (have 'int')
```

```
int a = 10;
int zero = a - 10;
int b = a / zero;
printf("%d\n", b);
// gcc: nothing reported; the program fails when it runs
```

5. The ErrorBuddy Language

ErrorBuddy compiles a small C-like language, kept small on purpose so that every phase could be completed and the effort could go into the error messages.

Category	Elements
Types	`int` only; every variable is declared with a starting value
Declaration	`int age = 20;`

Assignment	`age = age + 1;`
Output	`print expression;`
Operators	`+ - * /` with normal precedence; parentheses `( )`; integer division
Comments	`// to the end of the line`
Statement end	every statement ends with `;`
Not included	unary minus (write `0 - 5`), if/else, loops, functions, input

5.1 Grammar

```
program     -> statement*
statement   -> declaration | assignment | printStmt
declaration -> "int" IDENTIFIER "=" expression ";"
assignment  -> IDENTIFIER "=" expression ";"
printStmt   -> "print" expression ";"
expression  -> term (("+" | "-") term)*
term        -> factor (("*" | "/") factor)*
factor      -> NUMBER | IDENTIFIER | "(" expression ")"
```

6. System Architecture

The compiler is organised as five phases in three C++ source files. Phase 1 is in `lexer.cpp`. Phases 2, 3 and 4 run together in a single pass of `parser.cpp`: while a statement is recognised, its meaning is checked and its three-address code is emitted (syntax-directed translation). Phase 5, error reporting, is a shared channel used by every phase.

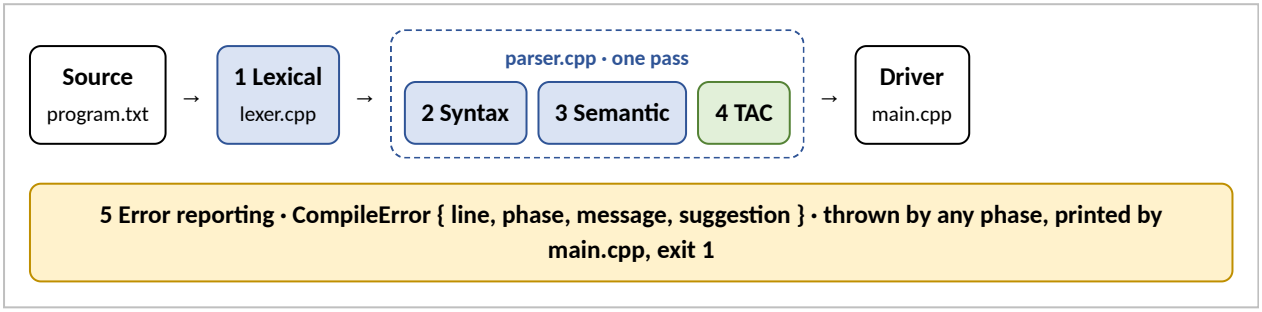


Figure 1: Architecture of the ErrorBuddy compiler. *Blue boxes are the standard front-end phases, green is intermediate code, yellow is the error-reporting phase that every other phase uses.*

6.1 Layers

Layer	Files	Role
Front end	lexer.h, lexer.cpp	Phase 1; defines Token and CompileError
One pass	parser.h, parser.cpp	Phases 2, 3 and 4 and the diagnosis helpers
Driver	main.cpp	Runs the phases, prints the result or the error, sets the exit code
Tools	Makefile, examples/, web/, site/, case_study/	Build, tests, playground, website, evaluation

6.2 Shared components

- **Token:** type, exact text and line number; created by the lexer and read by every later phase.
- **CompileError:** line, phase, message and suggestion; the only way any phase reports a problem.
- **Symbol table:** `std::map` from name to declaration line and current value.
- **TAC list:** instructions in source order, printed by the driver only when every phase passes.

6.3 Why the diagnosis sits inside the parser

The parser is the only place that knows the previous token, the next token and the rule being completed at the moment of failure. Moving the diagnosis there lets ErrorBuddy choose the line where the fix belongs. We measured the effect with a patched build that reports the detection point instead: it placed 21 of 24 mistakes on the exact line, against 24 of 24 for the shipped fix-point rule (Section 13).

Exit codes: 0 when every phase passes, 1 for a compile error, 2 for bad usage or an unreadable file, and 3 for an internal error.

6.4 Detailed Pipeline Flow (Stage-wise Architecture)

Figure 1A expands Figure 1 into the stage-by-stage data flow of the compiler. Arrows carry the artefact passed between stages, and red boxes mark where the pipeline halts with exit code 1.

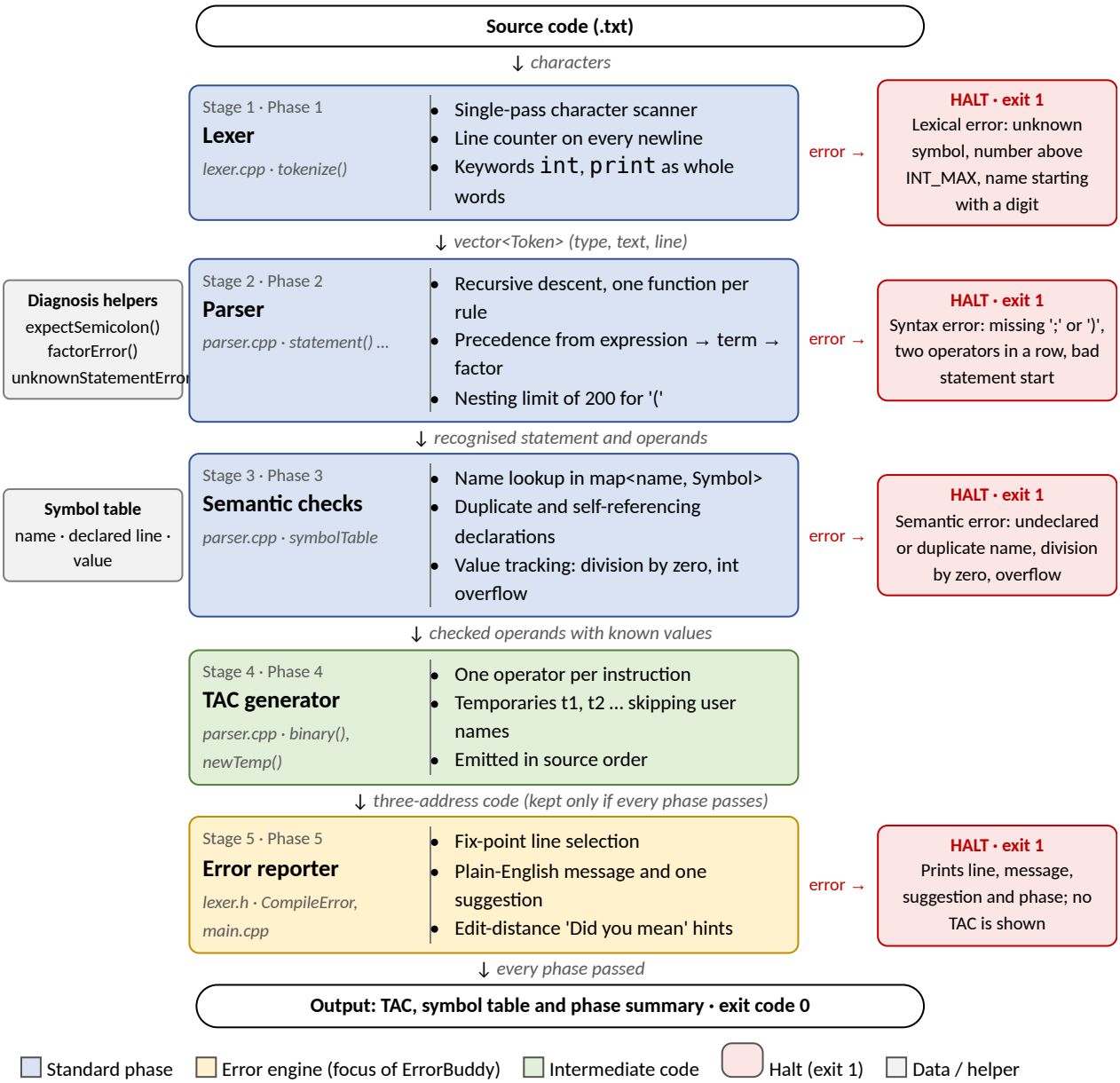


Figure 1A: Detailed stage-wise architecture of ErrorBuddy. *Stages 2 to 4 run in one pass of the parser. Stage 5 is entered from any earlier stage the moment a problem is found; it prints the error and no intermediate code.*

7. Phase-by-Phase Design

Phase 1: Lexical analysis

A hand-written scanner reads the source one character at a time. A newline increments the line counter, spaces and `//` comments are skipped, a run of digits becomes a NUMBER, a letter or underscore starts a name (the whole words `int` and `print` become keywords), and single characters become operators, '=', parentheses or ';'. The token list always ends with END_OF_FILE. It stops on an unknown symbol such as '@', a number larger than INT_MAX, or a name that starts with a digit such as '2nd'. Inspect with `--tokens`. Source: `lexer.cpp`.

```
Tokens (output of the lexer):
  Line  Type        Text
  1     KEYWORD     int
  1     IDENTIFIER  a
  1     ASSIGN      =
  1     NUMBER      5
  1     SEMICOLON   ;
  2     KEYWORD     int
  2     IDENTIFIER  b
  2     ASSIGN      =
  2     NUMBER      10
  2     SEMICOLON   ;
  3     KEYWORD     int
  3     IDENTIFIER  c
  3     ASSIGN      =
  3     IDENTIFIER  a
  3     OPERATOR    +
  3     IDENTIFIER  b
  3     OPERATOR    *
  3     NUMBER      2
  3     SEMICOLON   ;
  4     KEYWORD     print
  4     IDENTIFIER  c
  4     SEMICOLON   ;
```

Phase 2: Syntax analysis

A recursive-descent parser with one C++ function per grammar rule: `statement()`, `declaration()`, `assignment()`, `printStatement()`, `expression()`, `term()` and `factor()`. Precedence comes from the grammar: `term()` handles '*' and '/' one level below `expression()`, and the while loops in both make operators left-associative, so 10 - 4 - 3 is (10 - 4) - 3. `factor()` recurses on '(' up to a depth of 200. Stops on a missing ';' or ')', two operators in a row, a missing value, or a statement that starts with the wrong token. Source: `parser.cpp`.

Phase 3: Semantic analysis

The symbol table maps each name to its declaration line and current value. Checks run inside the parser the moment a name or operator is seen: undeclared variables (with a 'Did you mean' hint), duplicate declarations, a variable used in its own declaration, a keyword used as a name, division by zero and integer overflow. Because the language has no input and no loops, every value is known at

compile time, so a division by a variable that holds 0 is caught before the program runs. Source: `parser.cpp`.

Phase 4: Intermediate code generation

`binary()` computes the value of each operation, checks it, takes a fresh temporary from `newTemp()` and emits `tN = left op right`. Temporaries skip any name the user wrote, parentheses produce no instructions, and the code is emitted in source order and printed only if the whole program compiles. Source: `parser.cpp`.

```
Compilation successful!

Intermediate Code (Three-Address Code):
    age = 20
    score = 10
    t1 = age * 2
    t2 = score + t1
    score = t2
    t3 = score - 5
    t4 = t3 / 5
    bonus = t4
    print score
    t5 = age + 1
    t6 = bonus * t5
    print t6

Symbol Table:
    age         int   (declared on line 2)
    bonus       int   (declared on line 5)
    score       int   (declared on line 3)

Phases completed:
    [OK] 1. Lexical Analysis    - 41 tokens
    [OK] 2. Syntax Analysis     - 6 statements
    [OK] 3. Semantic Analysis   - 3 variables, no errors
    [OK] 4. Intermediate Code   - 12 instructions
```

Phase 5: Error reporting

Any phase throws a `CompileError` at the first problem. `main.cpp` catches it, prints the four fields in a fixed format and exits with code 1 (Section 8). Source: `lexer.h`, `parser.cpp`, `main.cpp`.

8. Error Reporting Subsystem

All errors travel through one structure, defined in `lexer.h` so that every phase can throw it:

```
struct CompileError {
    int line;
    std::string phase;       // which compiler phase found the problem
    std::string message;     // what is wrong, in plain English
    std::string suggestion;  // one concrete way to fix it
};
```

Field	Meaning	Example

line	where the fix belongs	2
phase	which phase found it	Syntax Analysis
message	the broken rule in plain English	Expected ';' at the end of the statement.
suggestion	one concrete thing to type	Add a semicolon after '10'.

Example messages produced by the real compiler:

```
Error on line 2:
Expected ';' at the end of the statement.
Suggestion: Add a semicolon after '10'.

(Found during: Syntax Analysis)
Compilation stopped. No intermediate code was generated.
```

```
Error on line 2:
Variable 'marks' has not been declared.
Suggestion: Declare it using 'int marks = 0;' before this line.

(Found during: Semantic Analysis)
Compilation stopped. No intermediate code was generated.
```

Errors stop compilation: the program is not partly compiled, and no intermediate code is printed. This keeps one message on screen, for one mistake, at the right line.

9. Diagnosis and Typo Hints

9.1 Diagnosis from surrounding tokens

`expectSemicolon()` runs at the end of every statement. If the next token is not ';', it checks the token in order:

Next token	Diagnosis	Reported on
')'	Found an extra ')' that has no matching '('.	that token's line
'='	Unexpected '='. A statement can contain only one '='.	that token's line
name, number or '(' on the same line	Missing operator between the two values.	that token's line
anything else	Expected ';' at the end of the statement.	**the previous token's line** (fix point)

`factorError()` handles a missing value: two operators in a row, a negative literal, empty parentheses, a keyword inside an expression, or a program that ends in the middle of an expression. `unknownStatementError()` handles a statement that starts wrongly: a misspelled or capitalised keyword, a name on its own, an extra ';' or an '=' with no name before it.

9.2 Typo hints

1. Compute the Levenshtein distance between the unknown word and each candidate, in a single rolling row of the dynamic-programming table.
2. Accept a candidate if the distance is at most 1 for targets of up to 3 letters and at most 2 for longer targets, and the word is not the candidate itself.
3. Skip variable names shorter than 3 letters, which produce misleading hints.
4. Offer the first accepted name: "Did you mean 'score'?".

The limits were chosen by testing four policies on 23 typo cases (`run_study.py --hints`):

Policy	Rule	Right	Wrong	Missed	Silent-ok
k1	1 edit, all names	10	8	3	3
shipped	1 edit if <=3 letters else 2, skip names <3	12	6	1	5
k2	2 edits, all names	13	9	0	2
k3	3 edits, all names	13	10	0	1

The shipped policy gives the fewest wrong suggestions (6) while still getting 12 right.

10. Sample Codes

10.1 Valid program: examples/valid.txt

```
// A valid ErrorBuddy program
int age = 20;
int score = 10;
score = score + age * 2;
int bonus = (score - 5) / 5;
print score;
print bonus * (age + 1);
```

Passes every phase: 41 tokens, 6 statements, 3 variables, 12 TAC instructions. Output shown in Section 7, Phase 4.

10.2 Missing parenthesis: examples/incorrect_parentheses.txt

```
int x = 2;
int y = (x + 3 * 4;
print y;
```

```
Error on line 2:
Missing ')' to close the '(' opened on line 2.
Suggestion: Add ')' after '4'.

(Found during: Syntax Analysis)
Compilation stopped. No intermediate code was generated.
```

10.3 Division by a variable that is zero: examples/division_by_zero.txt

```
int a = 10;
int zero = a - 10;
int b = a / zero;
print b;
```

```
Error on line 3:
Division by zero. Variable 'zero' is 0 at this point, so dividing by it is
impossible.
Suggestion: Make sure the value on the right of '/' is not zero.

(Found during: Semantic Analysis)
Compilation stopped. No intermediate code was generated.
```

gcc 11.4 compiles the equivalent C program without any warning.

10.4 Misspelled variable (case study M14)

```
int score = 10;
int bonus = 5;
scroe = score + bonus;
print score;
```

```
Error on line 3:
Variable 'scroe' has not been declared.
Suggestion: Did you mean 'score'? If 'scroe' is a new variable, write 'int scroe =
...;'.
```

All ten example programs are in the `examples/` folder of the repository with their expected output in `examples/expected/`.

11. Demo Screenshots

The screenshots were taken from the live GitHub Pages website. The playground uses the same page as the local playground (`make web`): the editor on the left highlights the error line, and the phase strip on the right shows how far compilation got.

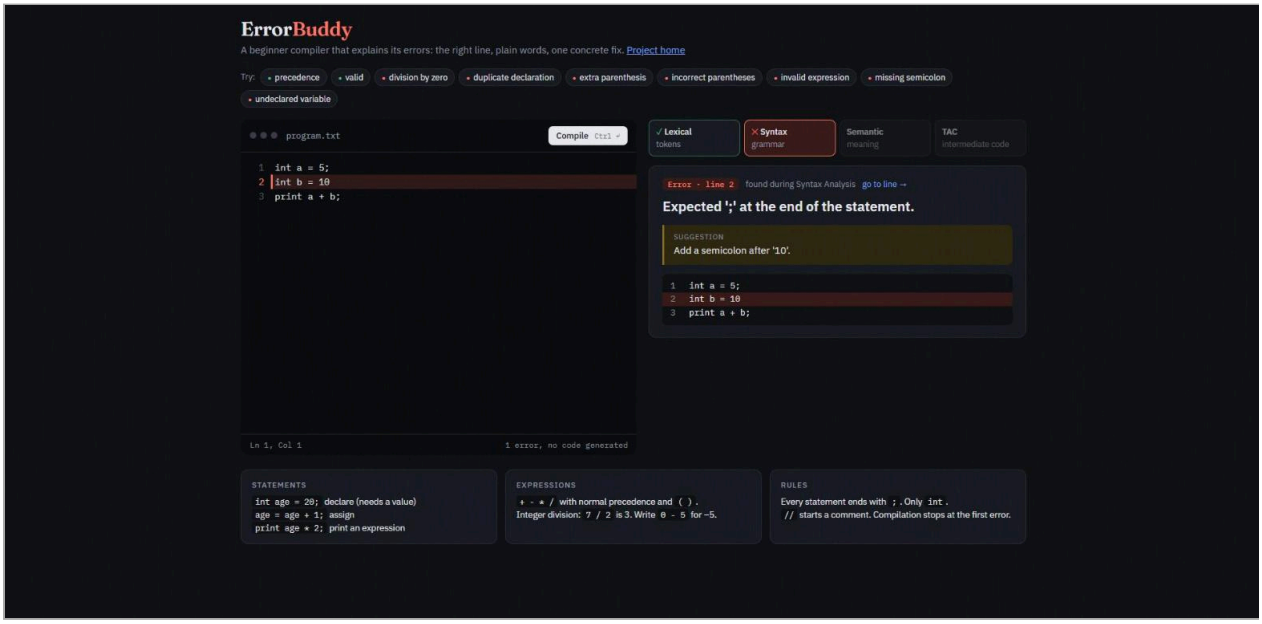


Figure 2: Missing semicolon in the playground. *The error is reported on line 2, where the fix belongs, with the suggestion "Add a semicolon after '10'". The phase strip shows Lexical passed and Syntax failed.*

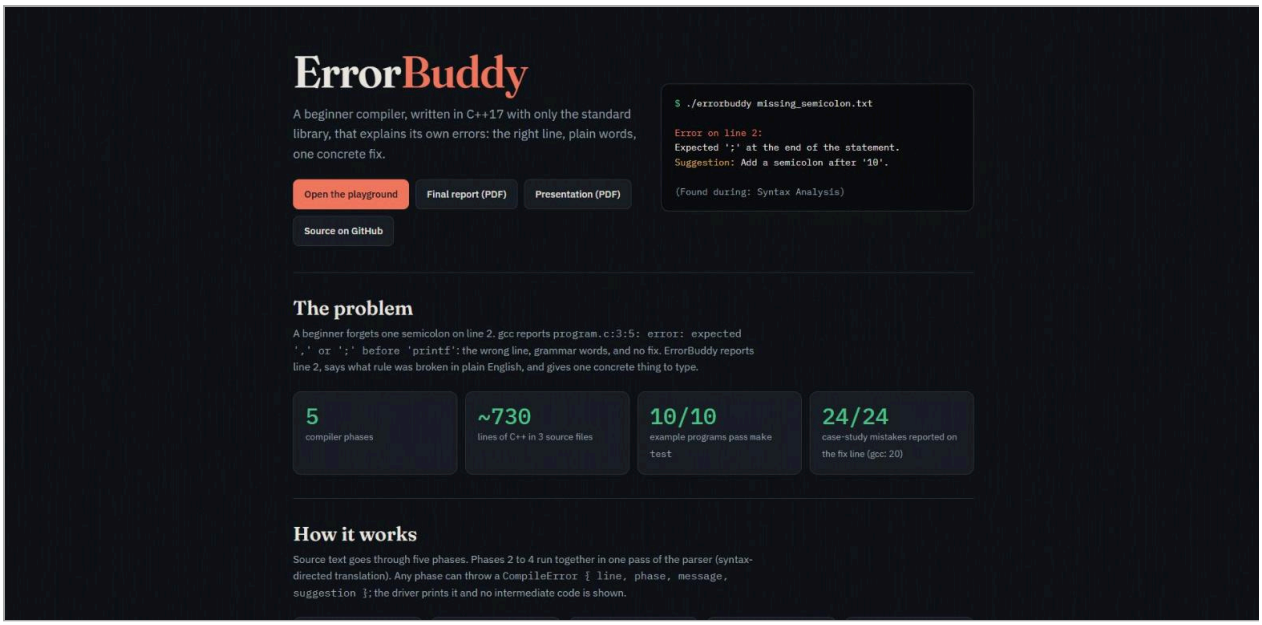


Figure 3: Project website on GitHub Pages. *Links to the playground, this report, the presentation and the source code.*

12. How to Start the Project

Repository (code and reference): https://github.com/Vardanp1/Compiler-Design-Project.git
Website: https://vardanp1.github.io/Compiler-Design-Project/

12.1 Prerequisites

- g++ with C++17 support (GCC 7 or newer) and GNU Make
- Python 3 (only for the web playground and the case study)
- gcc (only for the case-study comparison)

12.2 Clone and build the compiler

```
git clone https://github.com/Vardanp1/Compiler-Design-Project.git
cd Compiler-Design-Project
make
```

This builds `./errorbuddy` with `g++ -std=c++17 -Wall -Wextra -pedantic -O2`.

12.3 Run the compiler from the command line

```
./errorbuddy examples/valid.txt
./errorbuddy examples/precedence.txt --tokens
./errorbuddy examples/missing_semicolon.txt
make run FILE=examples/division_by_zero.txt
```

Command / flag	Purpose
`./errorbuddy file.txt`	Compile a file: TAC, symbol table and phase summary, or the error
`--tokens`	Also print the lexer output as a line / type / text table
`make test`	Run all example programs and compare with the expected output
`make web`	Start the local playground on http://localhost:8795
`python3 case_study/run_study.py`	Line-placement study against gcc (`--hints`, `--lexer` for the other studies)
`make clean`	Remove build files

12.4 Run the playground

```
make web
# open http://localhost:8795 in a browser
```

The local playground compiles anything you type by calling the real binary. The website playground shows the real compiler output for every example program.

13. Results and Comparison with GCC

The same 24 beginner mistakes were written in ErrorBuddy's language and as equivalent C programs. The C programs were compiled with gcc 11.4 (-Wall -Wextra, first diagnostic). The fix line of each case was labelled by hand before any compiler was run. ✗ marks a report on a line other than the fix line.

Mistake	Fix	gcc 11.4	ErrorBuddy
M01 missing ; (next line is print)	2	line 3: expected ',' or ';' before 'printf' ✗	line 2: Expected ';' at the end of the statement.
M02 missing ; (blank line + comment after)	2	line 5: expected ',' or ';' before 'printf' ✗	line 2: Expected ';' at the end of the statement.
M03 missing ; (next line is assignment)	2	line 3: expected ',' or ';' before 'b' ✗	line 2: Expected ';' at the end of the statement.
M04 missing ; on last statement	3	line 3: expected ';' before 'return'	line 3: Expected ';' at the end of the statement.
M05 missing)	2	line 2: expected ')' before ';' token	line 2: Missing ')' to close the '(' opened on line 2.
M06 missing) (expression split over lines)	3	line 3: expected ')' before ';' token	line 3: Missing ')' to close the '(' opened on line 2.
M07 extra)	2	line 2: expected ',' or ';' before ')' token	line 2: Found an extra ')' that has no matching '('.
M08 two operators in a row	2	line 2: invalid type argument of unary '*' (have 'int')	line 2: Two operators in a row: '+' followed by '*'.
M09 operator with no right operand	2	line 2: expected expression before ';' token	line 2: The operator '+' has no value on its right side.
M10 missing operator between values	3	line 3: expected ',' or ';' before 'b'	line 3: Missing operator between 'a' and 'b'.
M11 missing value after =	2	line 2: expected expression before ';' token	line 2: Missing value after '='.
M12 empty parentheses	2	line 2: expected expression before ')' token	line 2: Empty parentheses '()' are not allowed.
M13 undeclared variable	2	line 2: 'marks' undeclared (first use in this function)	line 2: Variable 'marks' has not been declared.
M14 misspelled variable	3	line 3: 'scroe' undeclared (first use in this function); did you mean 'score'?	line 3: Variable 'scroe' has not been declared.
M15 misspelled print keyword	2	line 2: implicit declaration of function 'prinft'; did you mean 'printf'? [-Wimplicit-function-declaration]	line 2: Unknown word 'prnt'.
M16 duplicate declaration	4	line 4: redefinition of 'age'	line 4: Variable 'age' has already been declared (on line 1).
M17 variable used in its own declaration	1	line 1: 'a' is used uninitialized [-Wuninitialized]	line 1: Variable 'a' is used in its own declaration, before it has a value.
M18 division by literal 0	2	line 2: division by zero [-Wdiv-by-zero]	line 2: Division by zero. You are dividing by the number 0.
M19 division by a variable that is 0	3	*nothing reported*	line 3: Division by zero. Variable 'zero' is 0 at this point, so dividing by it is impossible.

M20 division by an expression that is 0	1	line 1: division by zero [-Wdiv-by-zero]	line 1: Division by zero. The expression on the right of '/' works out to 0.
M21 arithmetic overflow	1	line 1: integer overflow in expression of type 'int' results in '-294967296' [-Woverflow]	line 1: The result of '*' here is too large to fit in an int.
M22 literal too large for int	1	line 1: overflow in conversion from 'long int' to 'int' changes value from '99999999999' to '1215752191' [-Woverflow]	line 1: The number 99999999999 is too large. An int can be at most 2147483647.
M23 unknown symbol	2	line 2: stray '@' in program	line 2: Unknown symbol '@'. ErrorBuddy only understands letters, digits and + - * / = ( ) ;
M24 name starts with a digit	1	line 1: invalid suffix "nd" on integer constant	line 1: '2nd' is not a valid name. Variable names cannot start with a digit.

Compiler	Caught	Exact fix line	Late	Early
gcc 11.4	23 / 24	20 / 24	3	0
ErrorBuddy, detection point	24 / 24	21 / 24	3	0
ErrorBuddy, fix point (shipped)	**24 / 24**	**24 / 24**	**0**	**0**

gcc missed one mistake entirely (M19, division by a variable that is 0), and all three of its late reports were a missing semicolon. Verification status: 10 of 10 example programs pass `make test`:

```
PASS  examples/division_by_zero.txt
PASS  examples/duplicate_declaration.txt
PASS  examples/empty.txt
PASS  examples/extra_parenthesis.txt
PASS  examples/incorrect_parentheses.txt
PASS  examples/invalid_expression.txt
PASS  examples/missing_semicolon.txt
PASS  examples/precedence.txt
PASS  examples/undeclared_variable.txt
PASS  examples/valid.txt
10 passed, 0 failed
```

14. Limitations and Future Work

14.1 Known limits

- The language has only `int`, arithmetic and `print`; there is no if/else, loop or function.
- Compilation stops at the first error, so a program with three mistakes needs three runs.
- There is no unary minus; `-5` must be written as `0 - 5`.
- Values are known at compile time only because there is no input and no loop.
- Three-address code is the final output; there is no optimiser or machine code.
- The website playground shows recorded output for the examples; compiling new code needs the local playground or the command line.

14.2 Future work

- if / else, while and for loops; comparison and logical operators; printf.
- Error recovery to report several errors per run.
- Functions and arrays; `char` and `float`.
- Data-flow analysis once loops and input exist.
- Constant folding on TAC and an assembly back end.
- A WebAssembly build so the website can compile any program in the browser.

15. Conclusion

ErrorBuddy shows that a small, hand-written compiler can implement every classic phase and still put the learner first. Treating each error as a structured object with a line, a phase, a message and a suggestion, and diagnosing the cause from the surrounding tokens before reporting, gives messages that point to the line where the fix belongs and say what to type. Measured on 24 classic beginner mistakes, it reported every one on the exact fix line, against 20 for gcc 11.4. Other compilers ask whether a program is valid; ErrorBuddy also answers where the mistake is and how to fix it.

16. References

1. Project repository: https://github.com/Vardanp1/Compiler-Design-Project.git
2. A. V. Aho, M. S. Lam, R. Sethi and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed., Addison-Wesley, 2006.
3. N. Wirth, *Compiler Construction*, Addison-Wesley, 1996.
4. V. I. Levenshtein, "Binary codes capable of correcting deletions, insertions, and reversals," *Soviet Physics Doklady*, vol. 10, no. 8, 1966.
5. V. J. Traver, "On compiler error messages: what they say and what they mean," *Advances in Human-Computer Interaction*, 2010.
6. G. Marceau, K. Fisler and S. Krishnamurthi, "Measuring the effectiveness of error messages designed for novice programmers," *Proc. ACM SIGCSE*, 2011.
7. P. Denny, A. Luxton-Reilly and D. Carpenter, "Enhancing syntax error messages appears ineffectual," *Proc. ITiCSE*, 2014.
8. B. A. Becker, G. Glanville, R. Iwashima, C. McDonnell, K. Goslin and C. Mooney, "Effective compiler error message enhancement for novice programming students," *Computer Science Education*, vol. 26, no. 2-3, 2016.
9. B. A. Becker et al., "Compiler error messages considered unhelpful: the landscape of text-based programming error message research," *Proc. ITiCSE Working Group Reports*, 2019.
10. GCC, the GNU Compiler Collection, version 11.4, warning options (-Wall -Wextra). https://gcc.gnu.org/onlinedocs/
11. Clang "Expressive Diagnostics" documentation. https://clang.llvm.org/diagnostics.html
12. E. Czaplicki, "Compiler errors for humans," Elm blog, 2015. https://elm-lang.org/news/compiler-errors-for-humans